

Forward Deployed Engineer, Onboarding Take-Home

Timeline & Expectations

You have **24 hours** from receiving this to submit your response. We expect the assignment to take approximately **2 hours of focused work** — we respect your time and care more about the quality of your thinking than the volume of output. Brevity is rewarded.

Note on Format

This take-home is designed to mirror the actual day-to-day of an Forward Deployed Engineer at AssemblyAI. You'll be context-switching between managing a portfolio of new startups building with AI and diving into a customer's integration to unblock them. The scenarios below are anonymized but reflect real situations our team handles regularly.

Resources

Use any tools you want, including AI tools, documentation, and internet research. We're as interested in *how* you work as in what you produce.

Part 1 — Monday Morning Pipeline

It's Monday morning. You have 5 in-flight leads in different states across your pipeline. You have ~45 minutes before your first customer call.

Your 5 leads

1. Plexus Health (plexushealth.com) — *Production customer*

- AI medical scribe for primary care clinics. Spending ~\$18k/mo on U-3 Pro Streaming.
- Datadog usage health alert fired for the third time in 60 days. Volume down 35% over the last 8 weeks.
- Their VP Eng (your champion) was responsive through Q1 but hasn't replied to your last two check-ins (Mar 22, Apr 11).
- Their CTO posted a "Speech Infrastructure Engineer" job two weeks ago.

2. Telegraph Studio (telegraph.studio) — *Self-serve, no human contact*

- Engineer signed up Apr 8 through the streaming docs.
- Made ~80k API calls in their second week, peaked at 14k in a single day on Apr 19.
- Zero traffic for the last 14 days.

- Domain points to a Series B-funded creative software startup; engineer's LinkedIn bio says "Real-time AI."

3. Veridis Routing (veridis.io) — Stalled active deal

- Real-time call-routing platform for insurance dispatch. Activated U2 streaming on contract in late January.
- ~50 hrs/day, has never grown. Still on U2 — never engaged with the U3 Pro migration push.
- Last 3 follow-up emails (Mar 1, Mar 18, Apr 9) unanswered.
- LinkedIn shows two new "Voice AI Engineer" hires at Veridis in the last 30 days, and you noticed a "Senior Speech Infrastructure Engineer" req on their careers page.

4. Northwell Bioacoustics (northwell-bio.com) — Stalled inbound from October

- Biodiversity monitoring platform — researchers dictate field observations alongside wildlife recordings; we'd transcribe the dictation, not the wildlife.
- Three discovery calls in November, said they'd launch "in Q1." Slip pattern since: "Q1" → "Q2" → most recent message (Apr 30): *"Still on track to begin integration this summer once our Series A closes."*
- Five rescheduled technical calls in four months. No API key, no test calls, no security review.
- PitchBook shows their last funding was a \$1.5M pre-seed Q3 last year; no record of an active Series A. Modest deal at scale (200–400 hrs/month if they ever launch).

5. Aniline Interpretation (aniline.health) — Hot inbound, three weeks in

- Real-time medical interpretation for hospitals — English/Spanish/Mandarin in noisy clinical settings. Just closed a \$4M seed; founders Sara Chen and Marcus Patel are both ex-Google Speech.
- After your intro call last week, they sent an 8-metric eval criteria doc — p95 latency <300ms, mid-utterance code-switching, HIPAA, WER on hospital noise. Evaluating Deepgram, Azure, and Speechmatics in parallel.
- Pilot scope: one NYC academic hospital + one CA rural hospital, 800–1,500 hrs/month. Mentioned a separate 50-hospital deal in flight that would 5× volume.
- Sara emailed yesterday: *"Bring data, not slides — we've been disappointed by every vendor's code-switching claims."* She also asked the call to include *"your applied AI lead, ideally someone who's worked on multilingual VAD."*
- 2GB of de-identified hospital audio is in your inbox; she expects transcripts before Tuesday's 11am call with both co-founders.

What we want from you:

For each lead:

- Your read on what's actually going on (a sentence or three)
- The next 1–2 moves you'd make and why

Then:

- How would you order your day across these five? Walk us through your reasoning — there's no single right answer, we want to see how you think.
 - Pick one lead and **write the actual outreach** you'd send (email, Slack DM, whatever channel makes sense for that lead).
-

Part 2 — DriveLine AI

You've been the FDE on the DriveLine AI account for the past two months. They're a drive-thru voice AI vendor; you ran their initial discovery, walked their lead engineer Marco through his first integration, and have a regular weekly call. The deal is in active commercial motion — our CEO Dylan has been parallel-tracking exec-level conversations with their CEO Raj, who's preparing a partnership proposal contingent on technical performance. Their pilot launch with a major QSR brand is 6 weeks out.

It's Wednesday evening. Marco just messaged you in your shared Slack channel:

Marco Jensen (DriveLine AI), Wed 6:47 PM — #aai-driveline

Hey — something's off with how our drive-thru sessions are closing. Datadog is full of `close_code: 3006` and our AAI bill doubled this month even though order volume only grew ~20%. Sending a longer email with our integration code now. Appreciate any eyes you can put on this tonight.

Ten minutes later, the email arrives:

From: Marco Jensen (Lead Engineer, DriveLine AI) **Subject:** Streaming integration — something's off with our session handling

Hey,

Two things have me worried this week:

1. About 48% of our streaming sessions show up in Datadog with `close_code: 3006`. These are sessions for actual orders that *had* audio — the transcripts come back fine during the order, but each session takes 5+ minutes to actually close after the customer is done.
2. Our AAI bill doubled between last month and this month, even though our actual order volume only grew ~20%.

Audio quality is fine, transcripts are accurate, end-of-turn detection works as expected during orders. It's specifically what happens *after* each order ends that I can't figure out.

Code is attached. We're piloting with [QSR partner] in 6 weeks for the summer rush and our CEO is asking hard questions about whether we should look elsewhere — Deepgram and Azure both have cleaner integration examples for our use case.

Marco

****The attached code is at the bottom of this doc****

What we want from you:

1. **Patched code** with brief comments on what you changed and why.
2. **A reply to Marco** — in Slack, email, or both, your call. What went wrong, how to fix it, what it means for his bill, anything you'd want him to clarify, and where this leaves things.
3. **A short internal note to AssemblyAI's Engineering / Product team** — what about our docs, SDK defaults, or onboarding flow led Marco to write the code this way, and what would you change to prevent the next customer from hitting this? This is the feedback loop Forward Deployed Engineers run for the rest of the company.

Bonus points for a Loom walking through your thought process end-to-end.

Submission

Send your response in whatever format best showcases your work. We're looking forward to seeing your approach!

Attached: `drive_thru_transcriber.py`

Python

```
"""
```

```
drive_thru_transcriber.py
```

```
FastAPI backend that opens an AssemblyAI streaming session per
drive-thru order. Each kiosk opens a WebSocket to /order when
a car pulls up; audio flows in, transcripts go to the POS system.
"""
```

```
import logging
```

```
import os
```

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
```

```
from assemblyai.streaming.v3 import (
    BeginEvent,
    StreamingClient,
    StreamingClientOptions,
    StreamingError,
    StreamingEvents,
    StreamingParameters,
    TerminationEvent,
    TurnEvent,
)
```

```
API_KEY = os.environ["AAI_API_KEY"]
```

```
app = FastAPI()
```

```
logger = logging.getLogger(__name__)
```

```
def on_begin(self, event: BeginEvent):
    logger.info(f"Session started: {event.id}")
```

```
def on_turn(self, event: TurnEvent):
    if event.end_of_turn and event.transcript:
        logger.info(f"Order utterance: {event.transcript}")
        send_to_pos_system(event.transcript)
```

```
def on_terminated(self, event: TerminationEvent):
    logger.info(
        f"Session terminated. "
        f"Audio: {event.audio_duration_seconds}s, "
```

```

        f"Session: {event.session_duration_seconds}s"
    )

def on_error(self, error: StreamingError):
    logger.error(f"Streaming error: {error}")

def make_streaming_client() -> StreamingClient:
    client = StreamingClient(StreamingClientOptions(api_key=API_KEY))
    client.on(StreamingEvents.Begin, on_begin)
    client.on(StreamingEvents.Turn, on_turn)
    client.on(StreamingEvents.Termination, on_terminated)
    client.on(StreamingEvents.Error, on_error)
    return client

@app.websocket("/order")
async def transcribe_order(websocket: WebSocket):
    """One WebSocket per car at the drive-thru."""
    await websocket.accept()

    aai_client = make_streaming_client()
    aai_client.connect(
        StreamingParameters(
            speech_model="u3-rt-pro",
            sample_rate=16000,
            format_turns=True,
            min_turn_silence=800,
            inactivity_timeout=300,
        )
    )

    try:
        while True:
            audio_chunk = await websocket.receive_bytes()
            aai_client.stream(audio_chunk)
    except WebSocketDisconnect:
        # Customer drove off - kiosk closed its end of the WebSocket.
        # We rely on inactivity_timeout to close the AAI session.
        logger.info("Drive-thru session ended (kiosk disconnected)")

```